



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Configuration Management, Integration, and Builds

Francesco Guerra
francesco.guerra@unimore.it

OBJECTIVES

- ▶ Describe the basic concept of configuration management.
- ▶ Analyze the relationship between the software artifacts and the development process.
- ▶ Discuss the configuration management framework.
- ▶ Describe the naming model of artifacts.
- ▶ Describe the storage and access model of artifacts.
- ▶ Describe the integration and build process.
- ▶ Discuss the various automation tools for configuration management.
- ▶ Analyze managing configuration management.

Software Configuration Management

- ▶ **Software configuration management is the process of managing all the pieces and parts of artifacts produced as part of software development and support activities.**
- ▶ It comprises:
 - ▶ Understanding the process activities, and the resulting outputs to be managed
 - ▶ Determining and defining the framework to be used to manage these outputs
 - ▶ Training and ensuring that the agreed-upon configuration management process is practiced

Policy, Process, and Artifacts

- ▶ The software development process determine the pieces to managed
- ▶ The chosen process and the outputs developed play a heavy role in the determination of what and what level of detail we must manage.
 - ▶ A conservative and risk-averse organization developing large and complex software may have a full **waterfall** process with inspections following every major activity such as requirements specification, design specification, coding, and test case generation.
 - ▶ A small and **agile** process-oriented organization may place emphasis on only a few activities, such as requirements specifications and coding. It may control only the final version of requirements specification and code.
- ▶ For example, an organization that adopts a conservative approach may decide on managing:
 - ▶ Requirements specification
 - ▶ Design specification
 - ▶ Source code
 - ▶ Executable code
 - ▶ Test cases

Policy, Process, and Artifacts (2)

► Version Management & Traceability

- Managing versions of individual artifacts is **tedious but not very complex**.
- Complexity increases when multiple artifacts, and the **relationships** among them, must be controlled.
- **Traceability** becomes a key issue: linking elements across different sets of artifacts (**inter-artifact relationships**).

► Examples of Inter-Artifact Relationships

- Requirements ↔ Test cases
- Design elements ↔ Code segments
- Code segments ↔ Database tables
- ...and many other possible links

► Types of Relationships

- **Input:** one artifact serves as the input for another.
- **Use:** one artifact is used or referenced by another.

An Example : Inter-Artifacts Relationship Matrix

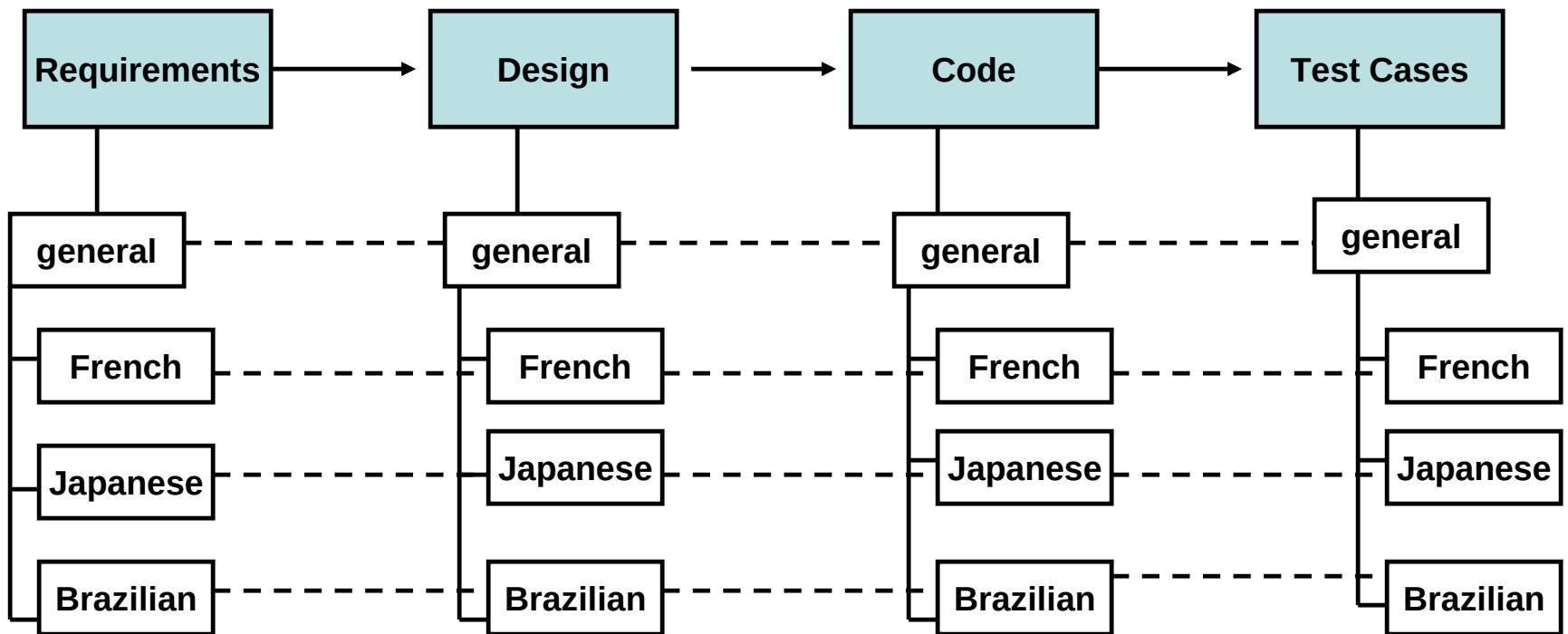
	Requirements Elements	Design Elements	Code Logic	UI Screens	DB tables	Initialization data	Test cases
Requirements Elements		X	X	X			X
Design Elements	X		X	X	X	X	
Code Logic	X	X		X	X	X	X
UI Screens	X	X	X			X	X
DB tables		X	X			X	
Initialization data		X	X	X	X		X
Test cases	X		X	X		X	

Policy, Process, and Artifacts (3)

- ▶ Complexity rises when **multiple versions** of each artifact must be managed.
- ▶ A software product may include:
 - ▶ Several versions of the **requirements**.
 - ▶ Slightly different software versions for **specific customers**.
 - ▶ **Country-specific** variants.
- ▶ **Intra-Artifact Relationships**
 - ▶ It may also be necessary to manage **relationships within the same artifact** (e.g., dependencies among requirements).
 - ▶ When **intra-artifact** and **inter-artifact** relationships must be considered simultaneously, an **extremely intricate network** of associations emerges.
- ▶ **Multi-Release and Multi-Country Complexity**
 - ▶ Managing multiple releases across multiple countries—each with different fixes and service packages—over several years further increases complexity.

“Multi -Relations: intra and inter entity”

- ▶ Intra entity - relationship: e.g. where there are multiple country versions of requirements
- ▶ Inter entity- relationship: e.g. where design is the input to code



Business Policy Impact on Configuration Management

- ▶ **Branching**
 - ▶ Branching duplicates the development or maintenance process so that modifications can occur in **parallel** on separate branches.
 - ▶ This requires a clear **policy for updates**.
- ▶ **Example Policy 1: Updates Only Through the General or Latest Version**
 - ▶ Requirements may be updated **only via the general version, or only on the latest version**.
 - ▶ Country-specific versions (e.g., French, Japanese, Brazilian) are then reviewed and updated **after** the general version changes.
 - ▶ This results in a **sequential update process**, since each country-specific version is considered a **successor** of the general one.
- ▶ **Example Policy 2: Country-Specific Versions Can Change Independently**
 - ▶ A country-specific version may be modified **independently** of the general version.
 - ▶ Example: *French version 2* is updated to *French version 2.1*.
 - ▶ When the general version is later updated (e.g., from version 2 to version 3), all version 3 requirements must be **reexamined** to ensure that the independent change to *French version 2.1* does **not conflict** with the new general version.

Configuration Management Framework: Naming Model

- ▶ To properly control all components, each artifact must be **uniquely identifiable**, and related items must be **associabile**.
- ▶ A consistent **naming model** can provide this structure.
- ▶ **Key Elements of the Naming Model**
 - ▶ A **unique product name** identifies the software.
 - ▶ There are multiple **artifact types** (e.g., design documents, source code, executables, requirement images, test scripts, test data).
 - ▶ Artifacts may exist in multiple **formats** (e.g., text documents, spreadsheets, images, audio clips).
- ▶ **Grouping and Versioning Scheme**
 - ▶ The scheme requires at least **two levels of grouping**:
 - ▶ **Software release level** — the primary classification.
 - ▶ **Version levels within a release** — multiple iterations may exist.
 - ▶ Additionally, the product may have **translated** or **country-specific** releases, which must also fit into the naming and grouping structure.

Configuration Management Framework

Naming Model (2)

- ▶ For this set of assumptions, the following six-part naming model may be used.

PP. CC. RRR. VVV. TT. FF

- ▶ Part 1—PP: It is the software product code. If P is allowed to be alphanumeric, then there are 36 characters for each position. Thus PP will give us 36^2 , or 1296, possible product codes.
- ▶ Part 2—CC: It represents the country code. Allowing 26 alphabet characters CC will allow 26^2 , or 676, countries.
- ▶ Part 3—RRR: It represents the release number of the software. If we use numerical figures 0 through 9, then RRR can represent 10^3 , or 1000, releases.
- ▶ Part 4—VVV: A three-positional versioning within each release, VVV is restarted for each release. Assuming only a numerical figure of 0 through 9, VVV would provide us with 1000 versions within each release.
- ▶ Part 5—TT: It describes the kind of artifact. It may be a requirements document, high-level design diagrams, database tables, test scenarios, test data, or source code. Here, T may be alphabetical and thus have 676 different types of artifacts.
- ▶ Part 6—FF: It represents the form the artifact is represented. The artifact may be in a document form, a spread sheet, in executable code form, or in a jpeg image form. F may be a numeric figure between 0 through 9. Thus FF provides 100 different types of forms.

Storage and Access Model

- ▶ The storage and access model defines the facilities needed to store and to control the access of all the software artifacts.
- ▶ Some fundamental functions:
 - ▶ create
 - ▶ delete
 - ▶ view
 - ▶ modify: It has to prevent the conflict and contention problems
 - ▶ return: to store the modified artifact
 - ▶ Modify and the return functions are paired.
 - ▶ Return function may include an automatic incrementing of the version number.
- ▶ These access functions provide a basic set of facilities for what is a version control

Storage and Access Model (2)

- ▶ **Functions in Existing Configuration Management / Version Control Systems**
 - ▶ **Import (Check-in):** store a new or updated artifact in the repository.
 - ▶ **Export (Check-out):** retrieve an artifact for editing or use.
 - ▶ **List:** display artifacts based on identifiers (e.g., product code, release number).
 - ▶ **Set:** assign a release number or version number.
 - ▶ **Increment:** increase the release or version number.
 - ▶ **Change:** modify identifying attributes (e.g., product code, country code, artifact type, artifact format).
 - ▶ **Gather:** collect all related artifacts by product code or release number.
 - ▶ **Merge:** integrate changes from one artifact or branch into another.
 - ▶ **Promote:** move selected artifacts to a different status (e.g., from development to testing).
 - ▶ **Compare:** identify differences between two versions of an artifact.

Storage and Access Model (3)

- ▶ Modern systems can provide other service functions to improve control, traceability, and collaboration.
- ▶ **Locking and Unlocking Mechanism**
 - ▶ Allows a user to **lock** an artifact while modifying it, preventing others from making changes. This avoid:
 - ▶ **Contention** (two users editing the same item at the same time)
 - ▶ **Conflicts** that require manual merging
 - ▶ Once the modification is complete, the artifact is **unlocked**. Some systems support **advisory locks** (warnings) or **mandatory locks** (exclusive access).
- ▶ **Where-Used / Where-Referenced Function**
 - ▶ Identifies **all artifacts that depend on or reference** a given artifact.
 - ▶ Essential for impact analysis:
 - ▶ When a requirement changes, it reveals which design documents, code modules, or test cases need updates.
 - ▶ When a database table changes, it shows which programs or queries are affected.
 - ▶ Supports better planning and reduces the risk of missing dependent updates.

Build and Integration

▶ The Build Process

- ▶ The build process consists of all activities needed to **integrate** and **convert** source files into executable components ready to run in a specific **execution environment** (hardware + software platform)

▶ Two Major Steps in the Build Cycle

- ▶ **1. Compile:** Each source file is translated into machine-readable form. If a compilation error occurs, the code must be corrected and **recompiled**.
- ▶ **2. Link:** The compiled modules are linked together. The linker resolves references to **external libraries**, shared components, and other required modules.

▶ Challenges in Large, Complex Systems

- ▶ Large applications may contain **hundreds of interdependent source files** that must be integrated correctly.
- ▶ These interconnected artifacts must be **properly controlled**, or the build process may fail—sometimes **after hours of processing**.

Build and Integration

▶ Common Issues in Multi-Developer Teams

- ▶ A frequent problem occurs when a developer modifies a source file **without informing the rest of the team.**
- ▶ Other modules depending on that modified file risk experiencing:
 - ▶ Reference mismatches
 - ▶ Interface inconsistencies
 - ▶ Linker failures
- ▶ These issues can cause the entire build activity to **crash**, wasting time and resources.

Build and Integration

- ▶ A special build file (**descriptor file** or **configuration file**) manages and controls all build activities. A well-known example from C and UNIX systems is the **makefile**, processed by the **Make** utility.
 - ▶ The build file defines and controls the sequence of essential build activities, such as:
 - ▶ **Dependency and cross-reference checking**
 - ▶ **Compilation** of source code into executables
 - ▶ **Linking** the executables with required external components
 - ▶ **Generating** any necessary data files
 - ▶ This ensures that the build process runs in the correct order and with consistent inputs.
- ▶ **Reliability and Traceability**
 - ▶ The build file helps ensure that if the current build fails, the team is **not left with nothing**.
 - ▶ It also acts as an **audit mechanism**, making it easier to trace the steps taken during the build.
 - ▶ Build errors can be linked back to specific steps or dependencies defined in the build file.
- ▶ **Incremental Builds**
 - ▶ For large systems, full build cycles can take **hours**, so builds are often **divided into separate parts**.
 - ▶ If not all components have changed, there is **no need for a full rebuild**.
 - ▶ Incremental builds improve productivity by recompiling and linking **only what has changed**.

Tools for Configuration Management

- ▶ Tier 1 : Version control and change control:
 - ▶ Revision control system (RCS)
 - ▶ Source code control system (RCCS)
 - ▶ Concurrent version system (CVS)
 - ▶ Apache SVN
 - ▶ Github
- ▶ Tier 2: Builds
 - ▶ Make utility
 - ▶ Odin
 - ▶ Cons
 - ▶ Ant
- ▶ Tier 3: Configuration Management for large systems
 - ▶ PVCS : ChangeMan (Serena Software)
 - ▶ Rational Clear Case (IBM)
 - ▶ Visual SourceSafe (Microsoft)
 - ▶ Perforce (Perforce Software)

Managing the Configuration Management Framework

▶ Software Configuration Management (SCM)

- ▶ Software configuration management is one of the areas that has **most successfully benefited** from advances in software engineering practices and research.

▶ Basic Role of SCM

- ▶ In its simplest form, SCM ensures that all source materials are properly managed so they can be:
 - ▶ **Compiled**
 - ▶ **Linked**
 - ▶ **Packaged** into a releasable product that users can install and execute.
- ▶ A core discipline of SCM is to:
 - ▶ Maintain a **clear record of multiple versions** of all source artifacts.
 - ▶ Deliver **any required version** for integration and build, enabling the creation of the correct software release for users.
 - ▶ Higher discipline in change control and traceability

Managing the Configuration Management Framework

▶ **SCM Administration**

- ▶ Many organizations designate a **configuration management administrator** to oversee these activities.
- ▶ This role ensures that processes are followed, artifacts are controlled, and releases are built consistently.

▶ **Extending SCM Beyond Source Code**

- ▶ Once source code and code-related artifacts are placed under SCM, an organization can expand the scope to include:
 - ▶ **Requirements**
 - ▶ **Design documents**
 - ▶ **Test cases**
 - ▶ **Other engineering artifacts**
- ▶ Managing these additional items—especially maintaining **proper relationships** among them—requires:
 - ▶ **More sophisticated configuration management techniques**
 - ▶ **Enhanced tooling and processes**



References

- ▶ Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Third Edition), Jones & Bartlett Learning

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.